

Appwrite & Firebase 하이브리드 환경 개발 / 운영 서버 분리 및 데이터 이관 실무 매뉴얼

운영 데이터 무손실 · 푸시 오발송 100% 방지 · 무중단 분리 아키텍처 가이드 (v1.1)

문서 버전: v1.1 (수정 반영)

대상 시스템: React + Appwrite + Firebase(FCM) 작성 목적: 완전 격리 개발환경 구축 및 안전 이관

1. 환경 분리 개요 및 필수 배경

현재 백엔드 데이터베이스는 **Appwrite**로 이관되었으며, 모바일/웹 푸시 알림 인프라는 **Firebase Cloud Messaging (FCM)**을 병행 사용하는 하이브리드 구성을 취하고 있습니다. 운영 중인 서비스에서 신규 기능을 개발하고 버그를 수정하기 위해서는 실제 사용자에게 **1) 데이터 오염/유실, 2) 서비스 중단(Downtime), 3) 개발 테스트 알림 오발송**이 절대 발생하지 않는 독립된 개발 공간이 필수적입니다.

⚠ FCM 하이브리드 환경 분리 시 최대 주의사항 (알림 오발송 위험)

Appwrite DB만 **Dev/Prod** 로 나누고 Firebase 프로젝트를 단일(공용)로 유지할 경우, 개발자가 로컬/개발 서버에서 결재 승인이나 알림 트리거를 테스트할 때 **실제 운영 중인 실사용자 스마트폰/브라우저로 테스트 푸시 알림이 발송되는 치명적인 사고**가 발생합니다. 반드시 **Appwrite와 Firebase 양쪽 모두 완전히 독립된 프로젝트로 1:1 분리**해야 합니다.

2. 권장 아키텍처: 듀얼 프로젝트 격리 모델 (Dual-Project Isolation)

서버 인프라 비용을 최적화하면서도 완벽한 샌드박스 환경을 구축하기 위해 **단일 Appwrite 호스트 서버 내 2개 프로젝트 분리 + Firebase 콘솔 내 2개 프로젝트 분리** 방식을 채택합니다.

[개발 환경 (Development Environment)]

프론트엔드 빌드 (.env.development / .env.local)

- Appwrite Project : **WorkFit-Dev** (DB 스키마 자유 수정 / 더미 데이터 적재)
- Firebase Project : **workfit-office-dev** (개발팀/테스터 단말기 FCM 토큰만 보관)

[운영 환경 (Production Environment)]

프론트엔드 빌드 (.env.production)

- Appwrite Project : **WorkFit-Prod** (실제 고객 데이터 / 엄격한 보안 권한)
- Firebase Project : **workfit-office-app** (실제 고객 단말기 FCM 토큰만 보관)

구분	개발 환경 (Dev)	운영 환경 (Prod)
목적 및 성격	신규 기능 구현, 파괴적 스키마 변경, API 테스트	안정적인 고객 실시간 서비스 운영 및 데이터 보존
Appwrite Project	workfit-dev (독립 DB, Storage, Auth)	workfit-prod (실제 운영 데이터 보관)
Firebase Project	workfit-office-dev (개발자 기기 토큰만 저장)	workfit-office-app (실사용자 기기 토큰만 저장)
알림 발송 격리	테스트 푸시 발송 시 개발자 폰에만 수신	실제 비즈니스 승인/알림만 실사용자에게 수신

3. 기존 통합 데이터 2단계 안전 마이그레이션 (Migration) 절차

기존 통합 환경의 데이터 유실 및 리스크를 원천 방지하기 위해 [1단계: 운영계 복원 및 정합성 검증] → [2단계: 개발계 샌드박스 복제]의 2단계 프로세스를 엄격히 준수합니다.

Step 1. [1단계] 운영계 복원 (Firebase → Appwrite Prod ETL 및 검증)

1. 원본 데이터 보존: 기존 Firebase (Firestore)의 실운영 원본 데이터를 백업합니다.
2. 운영 Appwrite(`workfit-prod`) 적재: ETL 스크립트를 통해 문서 ` \$id `와 계정 매핑을 유지하며 운영 DB에 적재합니다.
3. 정합성 검증: 전자결재 문서 수, 유저 권한, 첨부파일 링크의 정상 여부를 운영계에서 우선 교차 검증 완료합니다.

Step 2. [2단계] 개발계 복제 (Appwrite Prod → Appwrite Dev 마이그레이션)

검증 완료된 `workfit-prod` 의 스키마와 데이터를 `workfit-dev` 로 안전하게 복제하여 테스트 샌드박스를 구축합니다.

1. API Key 발급 (운영 프로젝트):

- 경로: `workfit-prod` > `Project Settings` > `API Keys` > `[Create API Key]`
- Scope(권한): `users.read` , `databases.read` , `collections.read` , `documents.read` , `files.read` , `buckets.read` 전체 체크

2. 마이그레이션 도구 실행 (개발 프로젝트):

- 경로: `workfit-dev` > `Settings` > `Migrations` > `[Create Migration]`
- Provider: `Appwrite` 선택
- 연결 정보: Endpoint URL, `workfit-prod` 의 Project ID, 1번에서 발급한 API Key 입력
- 복제 대상: Users, Databases (Collections & Documents), Storage 전체 체크 후 실행

⚠ 데이터 마이그레이션 시 체크리스트

1. 문서 고유 ID (\$id) 유지: 마이그레이션 도구는 기존 문서의 \$id 를 그대로 유지하여 전자결재 및 외래키 참조 관계를 보존합니다.
2. 컬렉션 접근 권한(Permissions) 재확인: 복제 후 각 컬렉션의 `Document Security` 및 역할별(Role) 읽기/쓰기 권한이 원본과 동일하게 켜져 있는지 확인합니다.

4. 프론트엔드 환경 변수 분기 및 빌드 설정 (React / Vite)

WorkFit 프로젝트 표준 명명법(`VITE_FB_*`)을 적용하고, 로컬 개발 시 실운영 DB 조작을 원천 방지하는 `.env.local` 가이드를 추가합니다.

1) 로컬 개발자 전용 파일 (`.env.local` - Git 제외 필수)

개별 개발자의 로컬 개발 머신에서 실행 시 무조건 개발계(Dev)를 바라보도록 강제합니다.

```
# 드라이버 모드 및 Appwrite DEV 설정
VITE_DB_DRIVER=appwrite
VITE_APPWRITE_ENDPOINT=https://appwrite.yourdomain.com/v1
VITE_APPWRITE_PROJECT_ID=workfit-dev-id
VITE_APPWRITE_DATABASE_ID=workfit-dev-db
|
# Firebase FCM DEV 설정 (VITE_FB_* 표준 명명법)
VITE_FB_API_KEY=AiZaSyDevApiKey...
VITE_FB_AUTH_DOMAIN=workfit-office-dev.firebaseio.com
VITE_FB_PROJECT_ID=workfit-office-dev
VITE_FB_STORAGE_BUCKET=workfit-office-dev.appspot.com
VITE_FB_MESSAGING_SENDER_ID=111122223333
VITE_FB_APP_ID=1:111122223333:web:abcdef123456
VITE_FB_VAPID_KEY=BDev_Vapid_Key_Example...
```

2) 개발 배포 환경 변수 파일 (`.env.development`)

```
VITE_DB_DRIVER=appwrite
VITE_APPWRITE_ENDPOINT=https://appwrite.yourdomain.com/v1
VITE_APPWRITE_PROJECT_ID=workfit-dev-id
VITE_APPWRITE_DATABASE_ID=workfit-dev-db
|
VITE_FB_API_KEY=AiZaSyDevApiKey...
VITE_FB_AUTH_DOMAIN=workfit-office-dev.firebaseio.com
VITE_FB_PROJECT_ID=workfit-office-dev
VITE_FB_STORAGE_BUCKET=workfit-office-dev.appspot.com
VITE_FB_MESSAGING_SENDER_ID=111122223333
VITE_FB_APP_ID=1:111122223333:web:abcdef123456
VITE_FB_VAPID_KEY=BDev_Vapid_Key_Example...
```

3) 운영 배포 환경 변수 파일 (`.env.production`)

```
VITE_DB_DRIVER=appwrite
VITE_APPWRITE_ENDPOINT=https://appwrite.yourdomain.com/v1
VITE_APPWRITE_PROJECT_ID=workfit-prod-id
VITE_APPWRITE_DATABASE_ID=workfit-prod-db
|
VITE_FB_API_KEY=AiZaSyProdApiKey...
VITE_FB_AUTH_DOMAIN=workfit-office-app.firebaseio.com
VITE_FB_PROJECT_ID=workfit-office-app
VITE_FB_STORAGE_BUCKET=workfit-office-app.appspot.com
VITE_FB_MESSAGING_SENDER_ID=999988887777
VITE_FB_APP_ID=1:999988887777:web:zyxwvu987654
VITE_FB_VAPID_KEY=BProd_Vapid_Key_Example...
```

4) Package.json 스크립트 분기

```
{
  "scripts": {
    "dev": "vite --mode development",
    "build:dev": "vite build --mode development",
    "build:prod": "vite build --mode production"
  }
}
```

5. FCM Service Worker 및 등록 로직 분기 처리

1) 서비스 워커 등록부 수정 (`src/shared/lib/messaging.ts`)

React 앱 번들 외부의 서비스 워커에 Vite 현재 모드를 쿼리 스트링으로 동적 전달합니다.

```
// src/shared/lib/messaging.ts
export const registerServiceWorker = async () => {
  if ('serviceWorker' in navigator) {
    const swUrl = `/firebase-messaging-sw.js?env=${import.meta.env.MODE}`;
    const registration = await navigator.serviceWorker.register(swUrl);
    console.log('[SW] Service Worker Registered with env:', import.meta.env.MODE);
    return registration;
  }
};
```

2) Firebase 콘솔 키 값 획득 경로 안내

서비스 워커 설정 객체(`devConfig` , `prodConfig`)의 키 값은 각 프로젝트 콘솔의 [Firebase 콘솔] → [프로젝트 설정(⚙️)] → [일반] → [내 앱] → [SDK 설정 및 구성 (구성 체크)]에서 발급받아 붙여넣습니다.

3) 서비스 워커 스크립트 (`public/firebase-messaging-sw.js`)

```
// public/firebase-messaging-sw.js
importScripts('https://www.gstatic.com/firebasejs/9.22.0/firebase-app-compat.js');
importScripts('https://www.gstatic.com/firebasejs/9.22.0/firebase-messaging-compat.js');

const params = new URLSearchParams(location.search);
const env = params.get('env') || 'production';

// Firebase 콘솔 > 프로젝트 설정 > 내 앱에서 획득한 설정값
const devConfig = {
  apiKey: "AIzaSyDevApiKey...",
  authDomain: "workfit-office-dev.firebaseio.com",
  projectId: "workfit-office-dev",
  storageBucket: "workfit-office-dev.appspot.com",
  messagingSenderId: "111122223333",
  appId: "1:111122223333:web:abcdef123456"
};

const prodConfig = {
  apiKey: "AIzaSyProdApiKey...",
  authDomain: "workfit-office-app.firebaseio.com",
  projectId: "workfit-office-app",
  storageBucket: "workfit-office-app.appspot.com",
  messagingSenderId: "999988887777",
  appId: "1:999988887777:web:zyxwvu987654"
};

firebase.initializeApp(env === 'development' ? devConfig : prodConfig);
const messaging = firebase.messaging();

messaging.onBackgroundMessage((payload) => {
  const notificationTitle = payload.notification.title;
  const notificationOptions = {
    body: payload.notification.body,
    icon: '/logo192.png'
  };
  self.registration.showNotification(notificationTitle, notificationOptions);
});
```

6. 개발 및 운영 워크플로우 & 듀얼 라이트 브리지 안전 수칙

🚨 [CAUTION] 듀얼 라이트 브리지 배포 및 에뮬레이터 구동 시 필수 교차 검증

현재 Firebase Cloud Functions 기반의 **듀얼 라이트 브리지(Dual-Write Bridge)**가 양방향으로 연동되어 있습니다. 개발용 Firebase Functions 배포(`firebase use workfit-office-dev`) 및 로컬 에뮬레이터 구동 시, **함수 실행 환경 변수가 운영용 Appwrite API Key나 Endpoint(workfit-prod)를 향하지 않도록 반드시 교차 검증**해야 합니다. 환경 변수 오설정 시 개발 테스트용 쓰기 작업이 운영 데이터베이스로 즉시 전파되어 실데이터가 오염될 수 있습니다.

🔧 개발팀 표준 작업 수칙 (Best Practices)

1. 로컬 개발 시: `.env.local` 이 로컬에 생성되어 있고 `VITE_DB_DRIVER=appwrite` 및 `workfit-dev-id` 가 지정되어 있는지 확인합니다.
2. 신규 기능 구현: 항상 `workfit-dev` 프로젝트를 바라보는 상태에서 DB 필드 추가, 인덱스 생성, 결재 로직을 검증합니다.

3. **푸시 테스트:** Dev 환경에서 알림을 수백 번 발송해도 `workfit-office-dev` 에 등록된 개발팀 테스트 단말기로만 수신됩니다.
4. **운영 배포 시:** 개발 완료 후 운영 Appwrite 컬렉션에 새 속성(Attribute)을 반영한 뒤, `npm run build:prod` 로 배포합니다.